In-Context vs. LoRA-Adapter Tool Calling as the Catalog Grows

A controlled comparison at 4, 16, and 64 tools on two models

Vitalii Nevidomyi

September 2026

Abstract

Tool-calling agents usually carry every tool schema in the prompt. This work measures what that costs at different catalog sizes and whether a per-catalog LoRA adapter can replace it. On 64 synthetic MCP-style tools with nested 4/16/64-tool catalogs, two models (GPT-OSS-20B; Inkling-Small, 276B/12B-active), three held-out phrasings (natural, seen paraphrases, unseen paraphrases from a disjoint glossaries), three training seeds, and tool-clustered bootstrap intervals. Findings: (1) Full-catalog prompting degrades with catalog size — GPT-OSS-20B scores 0.890 with one schema in context and 0.583 with 64, on the same queries; most of the loss is the correct tool with a wrong argument; tool's position in the schema block matters but less then tool itself (7% position vs. 80.5% for tool identity). (2) A word-overlap top-4 retriever recovers most of it 0.845 with no adapter (natural phrasing; 0.431 on unseen paraphrases). (3) An adapter trained on 50 natural + 50 paraphrased examples per tool, with no schema in the prompt, scores 0.951 on natural phrasing and 0.736 on unseen paraphrases at 64 tools — +0.368 [+0.313, +0.423] and +0.491 [+0.440, +0.541] over the full catalog, +0.106 and +0.305 over retrieval — at 152 input tokens instead of 7,251, and stays stable from 4 to 64 tools at 100 examples per tool. (4) Given the same data, fine-tuning with the schema in context is still better (though difference is small: +0.011 / +0.051 with one D seed at 64) while paying the full prefix price. (5) Adapter rank barely matters for natural phrasing: rank 1 (14M parameters) matches rank 32 within 0.01. It was shown before that adapter can beat a full-catalog promptin by Shemla et al. (2026); this work characterizes the gap against catalog size, retrieval, and matched with-schema fine-tuning.

Contents

1. Setup 2
2. Methods 3
    2.1 Tools and data 3
    2.2 Harness and scoring 4
    2.3 Training and schedule selection 4
3. Results 5
    3.1 The full-catalog penalty 5
    3.2 The adapter, across catalog size 7
    3.3 Fine-tuning with the schema vs. without, on the same data 8

3.4 Larger model at 64 tools . . . 9
3.5 Phrasing diversity . . . 10
3.6 LoRA Rank . . . 10
3.7 Economics — what was measured and what wasn't . . . 11
3.8 Scope: tool calls only . . . 11
3.9 The four options, side by side . . . 11
4. Related work and what this study adds . . . 12
5. Limitations . . . 13
6. AI assistance and author contributions . . . 14
Appendix A — the 8-seed grid at the pre-selection schedule . . . 14
Appendix B — per-seed results, final schedule . . . 14
Appendix C — the 12 tools hit hardest by the full catalog, GPT-OSS-20B . . . 15
References . . . 15

1. Setup

An agent with N tools normally carries all N schemas in every prompt. In this work we tried to replace it with LoRA adapter training instead emitting correct calls with no schema in context, and how does that compare as N grows.

Main Arms

arm	training	schema in prompt at eval
A	none	all N — the deployment baseline
B	none	none — the floor; confirms the tools are novel
C	LoRA SFT on (query → call) pairs	none
D	LoRA SFT with all N schemas in every training prompt	all N

Additional Arms

arm	schema in prompt at eval
single	schema in prompt is only for the target tool — used for diagnostic: argument-filling with no selection problem and a shorter prompt.
retrieve-k	prompt includes k schemas with the highest IDF-weighted word overlap with the query — the standard deployment alternative to the full catalog

Training recipes

- `ex100`: 100 naturally-phrased examples per tool.
- `mix50`: 50 natural + 50 paraphrased.
- `mix100`: 100 natural + 100 paraphrased.

Arm D was run with `ex100` at all sizes and with `mix50` at 4 and 64 tools (§3.3). Note: in our setup all training targets are tool calls, i.e.: there were no examples resulting in no call (§3.8).

Held-out sets. 50 queries per tool at each catalog size, all rewriting the same underlying held-out calls:

- **natural** phrasing;
- **seen paraphrases** (enum values described, not named, from a glossary that also generated the training paraphrases);
- **unseen paraphrases** (a second glossary, disjoint from the first).

Natural held-out accuracy was used to select the learning-rate schedule (§2.3), so it is a **validation set**, and natural-set numbers below are validation-set numbers. The unseen-paraphrase set was never used for any selection, it is a test set and the claims are stated on it. However it rewrites the same calls as the natural set, so is not fully independent of the selection either.

Models.

- `openai/gpt-oss-20b` for everything;
- `thinkingmachines/Inkling-Small` for the baselines and the `mix50` recipe at 64 tools.

Metrics.

- *Exact match*: single correct call, right tool, arguments equal after cosmetic string cleanup (§2.2).
- *Invention*: the call names the right tool but incorrect request. Exact formula is `tool_ok − schema_valid` which is **right tool, but the call fails schema validation** — an enum value not in the list, a parameter name that doesn't exist, a missing required parameter, or a wrong type.
- *Lenient*: exact match ignoring extra optional parameters.

Uncertainty. Where seeds are reported, ± is the population standard deviation across the training seeds run — variation over training runs, not over queries or catalogs. For the headline comparisons, 95% intervals come from a bootstrap that resamples *tools* (n = 64) with replacement after averaging seeds per tool, they are based on trained checkpoints and do not include training-seed uncertainty, which is why seed variability is reported separately.

2. Methods

2.1 Tools and data

Tools. 64 synthetic MCP tools generated by an LLM (one tool per domain: cave rescue, sourdough preparation, harpsichord tuning, etc.), so the base model has no prior on the exact vocabulary — arm B scores 0.000 at every size to confirm that. Parameters type mix fixed (40% enum, 20% bool, 20% int, 20% string), ≥1 enum per tool, 2–5 parameters, generic enum values banned. Catalogs are nested (4 ⊂ 16 ⊂ 64).

Queries. Generated by Grok 4.6 from randomly sampled ground-truth calls under a local checker (every value present, no snake_case tokens, no mention of omitted optionals). String values come from per-parameter pools **disjoint between train and held-out**. 150 queries per tool = 100 train + 50 held after validation.

Independent validation. Every query was re-read by a different model family (Gemini 3.8 Flash) with the full catalog in context and kept only if it reproduced the exact call and the request's intent class. The validator's rules were revised while inspecting its rejections at 64 tools — intent judged relative to the tool's own verb (a query_ tool's request is an inquiry), paraphrased enum values accepted, trailing punctuation ignored — and earlier verdicts were re-scored under the final rules (~1,900 changed, nearly all from the intent rule); tools still short of 100 train / 50 held were topped up with newly generated candidates. These revisions changed the validator, not any query's ground truth (calls are sampled before the text is written), and all of it preceded any evaluation of GPT-OSS or Inkling. Final pass rates: natural 99.1%, seen paraphrases 98.1%, unseen 97.8%. One consequence: every held-out query is one a strong model solved with the full catalog in front of it, so the base model's 0.583 at 64 tools is not a data-quality artifact.

Paraphrase sets. *Seen*: each natural query rewritten so no enum value's words appear, enum description was selected from per-tool glossary of validated paraphrases (zero word overlap with the value). *Unseen*: the held-out queries rewritten with a second glossary generated separately and not reusing any of first-glossary paraphrases: 465 of 465 enum values received a distinct second paraphrase. The unseen paraphrases are not used for any adapter training.

2.2 Harness and scoring

Prompting and parsing go entirely through the `tinker-cookbook` renderer — `gpt_oss_low_reasoning` for GPT-OSS (the recommended `gpt_oss_no_sysprompt` produced 8.5% malformed tool-call envelopes and 3.5% reasoning cap-outs on this task, reported separately as a cookbook issue), `tml_v0` for Inkling-Small. Temperature 0, `max_tokens` 1024, renderer stop sequences, same developer instruction.

Scoring. Exact-match did cosmetic fixes for digit-strings and booleans to match schema types and trimmed surrounding whitespace and sentence-final punctuation from strings. Re-scoring all 502,110 evaluated responses with no normalization changed 22 outcomes (0.004%).

Repeat variability. Two base-model runs on byte-identical prompts at temperature 0 disagreed on 4 of 200 queries — 2% in range of batch noise. Also one-token change to the rendered date produced 7 disagreements with the first run (5 regressions, 2 improvements; exact two-sided McNemar $p = 0.45$), not distinguishable from that repeat variability. Most runs carried current date before it was pinned.

The harness and pipeline scripts were written with AI assistance (§6).

2.3 Training and schedule selection

LoRA via Tinker, rank 32 was used for anything but rank specific runs, batch is 16 sequences, 3 epochs, 10% linear warmup, loss on the assistant part of prompt only (SFT).

Rule for selection: on the natural held-out set, picked the lowest-σ setting among those within 0.02 of the best mean; ties broken by mean. Applied at 4 tools (constant

1e-4 / 2e-4 / 5e-4; decayed 5e-4), re-applied at 16 (adding decayed 2e-4 and 1e-3), checked at 64:

- **Constant LR drifts.** At 5e-4 constant (4 tools), loss reaches its floor by step 20 and wanders at full LR for 55 more; final-checkpoint σ across seeds 0.04–0.12; unseen accuracy 0.08–0.11 below any decayed setting. At 2e-4 constant (16 tools), no settled tail through the last step; σ 4× wider.
- **Decay fixes it; base LR then matters little.** Under linear decay, 2e-4 / 5e-4 / 1e-3 land within 0.025 at 16 tools; 1e-3 is the upper edge; 2e-4 undertrains at 4 tools (75 steps). 5e-4 decayed was never worse at any size.
- **Tail loss predicts seed variance without touching held-out data.** Max loss/token over the final 10% of steps: 0.026 for the drifting run, ≤ 0.011 for every settled run.
- **Checked at 64 tools** (1,200 steps): `mix50` at 2e-4 decayed 0.948 / 0.710 vs 5e-4 decayed 0.951 / 0.736 (natural / unseen, 3 seeds each).

Final schedule: 5e-4, linear decay. Seed SD on natural 0.001–0.02 at every size. The first grid (2e-4 constant, 8 seeds at 4 tools) is in Appendix A.

3. Results

3.1 The full-catalog penalty

Base model, all schemas in the prompt, GPT-OSS-20B:

tools	closed vocab	schema tokens	natural	unseen	tool_ok	invention
4	55	512	0.865	0.422	1.000	0.5%
16	177	1,703	0.775	0.403	0.951	0.7%
64	732	7,099	**0.583**	**0.243**	0.891	5.2%

To check if it tools harder or is it the prompt? Same queries, with only the query's schema in context:

GPT-OSS-20B, 64 tools, same queries (3,200 natural / 3,122 unseen)	natural	unseen
single schema	**0.890**	0.566
all 64 schemas	0.583	0.243
penalty	-0.307	-0.323

One at a time, the model reads these 64 tools as well as it read the pilot's 4. The penalty is coming from catalog size. Prompt length increase, with that the number of distractors.

Decomposed by failure class:

failure, same queries	single	all 64	Δ
no call emitted	0.7%	8.5%	+7.8
wrong tool	0.0%	2.4%	+2.4
right tool, **invented** enum	0.5%	5.2%	+4.7
right tool, valid schema, **wrong value**	9.8%	25.6%	**+15.8**

Most of the penalty is the correct tool with wrong arguments. Note: a wrong tool is rarely emitted does not show that interference from other schemas contributes little to the argument errors — only that it rarely surfaces as a selection error.

Position in the schema block does matter, but tool identity matters more. Schema order was checked on four rotations (k=0, 16, 32, 48 positions; rotation k moves the tool at index i to position (i + k) mod 64) so every tool occupied every quarter of the rendered block exactly once, plus two random shuffles (seeds 1 and 2) as a sensitivity check. Rotation-based variance use only the four rotations; the per-order correlations use all six. Pooled over four rotations base-model accuracy by quarter is **0.691, 0.603, 0.531, 0.533**; the penalty relative to single-schema is +0.20, +0.29, +0.36, +0.36. Tools rendered in the first sixteen slots lose 0.20 to the full catalog; in the back half, 0.36; the effect saturates after the midpoint. The within-order rank correlation between position and penalty is positive under all six orders (ρ = +0.17 to +0.41). In a variance decomposition over the four rotation runs (64 tools × 4 orders = 256 cells of per-tool accuracy; the design is balanced, so the tool-identity and position-quarter sums of squares are additive; quarter = ⌊position × 4 / 64⌋), **tool identity accounts for 80.5%, position quarter 7.0%, rest 12.5%** (the quarter term is identical whether computed directly or after removing tool means). Position is a real, secondary effect on average and a large one for some tools (the same tool's accuracy moved 0.25 on average across the four rotations; `adjust_mist_net_tension` scored 0.50 and 0.44 in the front half, 0.12 and 0.16 in the back). Overall accuracy varied from 0.564 to 0.602 across six orders — a 0.04 swing from a choice of order. Parameter count predicts the penalty on both models (ρ = +0.31 / +0.36); lexical similarity to the nearest other tool does not (ρ = +0.12 / +0.17).

covariate vs. penalty (Spearman ρ, 64 tools)*	GPT-OSS-20B	Inkling-Small
number of parameters	+0.31	+0.36
closed vocab (per tool)	+0.11	+0.14
lexical similarity to nearest other tool	+0.12	+0.17
tools sharing the verb	−0.02	+0.07
position in the original order	+0.41	+0.08

*all from original order (rotation 0)

Schema retrieval recovers most of it. Supplying only the k schemas with the highest word overlap with the query:

GPT-OSS-20B, 64 tools	natural	unseen	recall@k* natural / unseen	input tokens
full catalog	0.583	0.243	—	7,251
lexical top-4	**0.845**	**0.431**	0.968 / 0.846	645
lexical top-8	0.839	0.425	0.979 / 0.900	1,106
single schema (oracle top-1)	0.890	0.566	1.0 / 1.0	294

*how often selected tool set had target tool in it.

Simple word-overlap retriever with no training takes natural-phrasing accuracy from 0.583 to 0.845 at a tenth of the tokens; k = 8 adds nothing. On paraphrases the retriever itself misses 15% of the time at k = 4 paraphrases exclude words it suppose to match on, so accuracy drops. For a deployment decision top-k retrieval is the baseline the adapter has to beat.

3.2 The adapter, across catalog size

Final schedule (§2.3), 3 seeds, GPT-OSS-20B. Natural / unseen paraphrases, ± = seed SD:

	4 tools	**16 tools**	**64 tools**
A — full catalog	0.865 / 0.422	0.775 / 0.403	0.583 / 0.243
C — ex100, no schema	0.955±.011 / 0.345±.015	0.970±.003 / 0.479±.013	0.953±.002 / 0.416±.013
D — ex100, schema	0.982±.006 / 0.438±.063	0.982±.008 / 0.603±.015	0.975 / 0.571 [1]
C — mix50, no schema	0.933±.019 / 0.650±.021	0.960±.007 / 0.734±.021	**0.951±.001 / 0.736±.015**
C — mix100, no schema	0.957±.012 / 0.718±.047	0.975±.006 / 0.793±.005	**0.969±.005 / 0.759±.015**
D — mix50, schema	0.977±.012 / 0.748±.008	—	0.962 / 0.786 [2]

[1] one seed, one epoch. Per-seed values in Appendix B.

[2] run for 16 tools was never done and only single seed for 64.

Adapter vs. full catalog at 64 tools. For each of the 64 tools, we compute the adapter's accuracy (mix50, averaged over its three seeds) minus the base model's accuracy with all 64 schemas in the prompt, on that tool's held-out queries. The headline Δ is the mean of these 64 per-tool differences. The 95% interval comes from a bootstrap that resamples whole tools — draw 64 tools with replacement, recompute the mean, 10,000 times, and take the 2.5th and 97.5th percentiles. Tools rather than queries are resampled because a tool's queries are not independent of one another. This Δ weights each tool equally. The model-comparison table in §3.4 pools all queries instead, so tools with more retained held-out queries count slightly more; that is why its unseen gain reads +0.493 rather than +0.491.

	Δ (tool-weighted)	95% CI	tools with Δ > 0
natural	**+0.368**	+0.313 to +0.423	64 of 64
unseen paraphrases	**+0.491**	+0.440 to +0.541	64 of 64

Against the realistic alternative: lexical top-4 retrieval, the adapter's margin is **+0.106 on natural and +0.305 on unseen** (query-weighted; 0.951 vs 0.845, 0.736 vs 0.431), at 152 input tokens against 645. Paired by tool, 95% tool-clustered intervals: natural +0.106 [+0.072, +0.144] (better on 45 of 64 tools), unseen +0.304 [+0.259, +0.348] (better on 60 of 64 tools). The natural-phrasing margin is modest; the paraphrase margin is larger. A retrieval step that misses 15% of paraphrased queries (recall@4 = 0.85) is one candidate explanation; it is not tested here.

Stable over 4–64 tools, with training data scaling per catalog. `mix50`: 0.933 → 0.960 → 0.951 on natural as the vocabulary grows 55 → 732 symbols. Each catalog's adapter trained on 100 examples per tool, so total training data grew 16× as well. Stability here is under a fixed per-tool data budget.

Invention doesn't rise with vocabulary. On unseen paraphrases:

- `ex100` 47% → 42% → 47%;
- `mix50` 15% → 13% → 11%;
- D — `ex100` 4% → 7% → 4%;
- A 1.5% → 0.6% → 5.6%.

3.3 Fine-tuning with the schema vs. without, on the same data

Arm D: the schema in every training prompt and at evaluation. Was run with `mix50` data matched to C on data, 3 epochs, same schedule, 3 seeds at 4 tools and one for D and three for C at 64 tools.

same `mix50` data, 3 epochs	natural	seen paraphrases	unseen	invention (unseen)	input tokens
4 tools — C, no schema	0.933 ± .019	0.902 ± .017	0.650 ± .021	15%	151
4 tools — D, schema	**0.977 ± .012**	**0.938 ± .012**	**0.748 ± .008**	**0.8%**	663
Δ (D − C)	+0.044	+0.036	+0.098		4.4×
64 tools — C, no schema	0.951 ± .001	0.931 ± .010	0.736 ± .015	11%	152
64 tools — D, schema [1]	**0.962**	**0.946**	**0.786**	**1.7%**	7,251
Δ (D − C)	+0.011	+0.015	+0.050		48×

[1] one seed; D's training-run variability at 64 tools is unknown

D exceeds each of C's three seeds on natural and unseen accuracy. Paired tool-clustered 95% intervals for D − C, on these checkpoints: natural +0.011 [+0.0004, +0.021] (D better in 36 of 64), unseen +0.051 [+0.029, +0.073] (D better in 42 of 64).

Given identical training data, keeping the schema in the prompt is better on every measure at 4 tools (three seeds each). At 64 tools, with one D seed, the estimated advantage on unseen paraphrases is +0.051 (interval +0.029 to +0.073, 42 of 64 tools) and on natural phrasing +0.011 (interval +0.0004 to +0.021, better for 36 of 64 tools). Invention on paraphrases is 1.7% against C's 11%. **The schema-free configuration — no schema at training or evaluation — did not generalize better than the schema-present configuration at either size, given the same data.** The natural-only comparison (`ex100`) points the same way at every size.

The observed advantage was smaller at 64 tools than at 4: D − C on natural phrasing +0.044 vs +0.011, on unseen paraphrases +0.098 vs +0.050, while D's prompt grew from 663 to 7,251 tokens. Both arms improved with the larger catalog (C's unseen accuracy 0.650 → 0.736, D's 0.748 → 0.786); the two sizes differ in tools, queries, training-set size, and checkpoints, the cause of the smaller gap is not identified. Note also that C and D differ in the schema's presence at both training and evaluation, so the comparison does not isolate removing the schema from training alone. The schema-free adapter's case is the input-token tradeoff: at 64 tools it gives up about 0.01 on natural phrasing and 0.05 on paraphrases — and its invented-enum rate on paraphrases is 11% against D's 1.7% — for 48× fewer input tokens per request.

3.4 Larger model at 64 tools

	GPT-OSS-20B	Inkling-Small
full catalog (A), natural / unseen	0.583 / 0.243	0.781 / 0.552
lexical top-4 retrieval, natural / unseen	0.845 / 0.431	not run
single-schema baseline, natural / unseen	0.890 / 0.566	0.897 / 0.782
mix50 adapter, natural / unseen	**0.951±.001 / 0.736±.015** (n=3)	**0.959±.002 / 0.827±.004** (n=2)
adapter − full catalog (query-weighted)	+0.368 / +0.493	+0.178 / +0.275
adapter − single-schema	+0.061 / +0.170	+0.062 / +0.045
input tokens, full catalog → adapter	7,251 → 152 (47.7×)	9,481 → 95 (99.8×)

The two models read a single tool about equally well on natural phrasing (0.890 vs 0.897). Under the full catalog the larger model loses 0.12, the smaller 0.31. **Adapter accuracy on natural phrasing was similar (0.951 vs 0.959); paraphrase generalization was model-dependent** — 0.736 vs 0.827 with the adapter, and 0.566 vs 0.782 with a single schema. The larger model's advantage appears both in long prefixes and in paraphrase mapping.

The adapter exceeds each model's single-schema baseline by ~0.06 on natural phrasing. That baseline is a base-model number; fine-tuning also fixes reading errors the

base model makes with one tool in front of it, concentrated on the tools it reads worst: the 9 (GPT-OSS) and 12 (Inkling) tools the base model gets below 0.75 alone score ~0.90 with the adapter.

Inkling-Small fills unmentioned optional parameters despite the instruction; lenient scoring puts it at 0.985 at 4 tools vs 0.865 exact. Its wrong-tool errors are mostly hallucinated names (`flag_mortality_event → report_mortality_event`, 40 of 41), this is different failure from GPT-OSS's errors (like`adjust_typebar_strike → adjust_press_tension`).

3.5 Phrasing diversity

Natural-only training learns the call format: "low oxygen" → `low_oxygen` is transcription. Which leads it, on unseen phrasing, to generate enum values that are not in the schema: "the air's gone thin" → `thin_air`. ex100 sits low: at 0.35–0.48 on unseen paraphrases at every size with ~45% invention.

Fifty paraphrased examples per tool improve unseen to 0.65-0.74 and cut invention to 11-15%; a hundred only adds 0.02-0.07 to that. On natural phrasing diversity at training help too (ex100 0.953 → mix100 0.969 at 64 tools, invention 2.2% → 0.7%).

Seen-to-unseen transfer gap. mix50 at 64 tools: 0.931 on paraphrases from the glossary it trained on, 0.736 on paraphrases from a glossary it never saw. The 0.20 gap may reflect familiarity with the training glossary, glossary difficulty, or both: the *base model*, which saw neither glossary, also scores lower on the second (4 tools: 0.558 seen vs 0.422 unseen; 64 tools: 0.292 vs 0.243). This split is not measured here.

3.6 LoRA Rank

mix50, 64 tools, 3 seeds:

rank	adapter size	natural	seen paraphrases	unseen	invention (unseen)
1	29 MB (~14M params)	0.946±.001	0.926±.004	0.682±.006	15.4%
4	114 MB	0.945±.005	0.922±.003	0.701±.006	14.3%
8	228 MB	0.946±.003	0.923±.003	0.721±.009	12.5%
32	913 MB	0.951±.001	0.931±.010	0.736±.015	11.4%

Natural and seen: within 0.006 and 0.009 across 32× in parameters. Unseen: roughly +0.01 per doubling of rank, flattening. Increasing rank helped unseen accuracy more than natural accuracy under this schedule. Within one adapter, per-tool unseen accuracy correlates with the tool's vocabulary size (ρ = -0.56). Consistent with Thinking Machines' LoRA-Without-Regret findings, in a domain they didn't test.

This is a single shared adapter holding the whole catalog. ParaTool (Yu et al., 2026) reports that a single shared LoRA over all tools (their "Global Parameterization" base-

line) performs poorly on Stable ToolBench and BFCL, and proposes per-tool modules with a gating network instead. Shared-adapter performance evidently differs between the two studies; they also differ in models, data, training recipe, task, and the number of candidate tools presented per request, none of those causes has been tested.

3.7 Economics — what was measured and what wasn't

Measured: input tokens per request drop from 7,251 to 152 (GPT-OSS, 47.7×) and from 9,481 to 95 (Inkling-Small, 99.8×). Against top-4 retrieval the drop more modest 645 → 152 (4.2×). Output tokens were similar or lower for the adapter (37–41 vs 65–78). Adapter training cost ~$1.50 (GPT-OSS) / ~$4.60 (Inkling-Small) at 64 tools; D's training cost with the schema in every example ~$19 for one epoch (~$56 for three).

Not measured: total request cost, which depends on caching, output tokens, adapter-serving overhead, and serving configuration; it is not proportional to input tokens even without caching. In the 64-tool full-catalog base-model runs, `prompt_cache_hit_tokens` averaged 7,166 of 7,251 prompt tokens (GPT-OSS) and 9,442 of 9,481 (Inkling-Small) — the schema prefix was almost entirely served from cache within each run.

KV-cache footprint, calculated from the published GPT-OSS-20B configuration (24 layers, 8 KV heads, head dim 64, bf16): 48 KiB per token, ~349 MB for 7,099 schema tokens if every layer attended fully is an upper bound, since half the layers use a 128-token sliding window. This is an architectural calculation, not a measurement of Tinker's serving memory.

3.8 Scope: tool calls only

The adapter was trained on tool calls only — no "no tool applies" examples — because the question was whether an adapter can hold a schema, and narrowing the output to the one task is the design chosen for that question. The consequence: adapters answered every general question (not a tool call) with a tool call. Rank 32 mostly reused a few invented generic names (`query_choice` 37 times, `list_options`); rank 1 composed a plausible name per question (`query_bit_shift_left` for a computer-science item, `flag_straw_man_defension` for a fallacies item) or filled a real tool's arguments with the question text. This is the expected behavior of a model whose training targets were 100% tool calls, and it is the same compose-a-name behavior seen within-domain as invention (§3.5) and in Inkling-Small's hallucinated tool names (§3.4).

3.9 The four options, side by side

For a fixed 64-tool catalog on GPT-OSS-20B, ordered by input tokens per request:

option	training	natural	unseen	input tokens	notes
fine-tune, no schema (C, mix50)	~$1.50	0.951	0.736	152	fewest tokens
top-4 lexical retrieval	none	0.845	0.431	645	recovers most of the full-catalog loss with no training; misses 15% of paraphrases
fine-tune, schema in context (D, mix50)	~$56	0.962 [2]	0.786 [2]	7,251	highest observed accuracy; fewer invented enums than C; pays the full prefix
full catalog in context	none	0.583	0.243	7,251	worst in this setting; position-sensitive

[2] one seed.

The C–D trade-off, on the same training data: D's observed accuracy is higher by 0.011 on natural phrasing (interval +0.0004 to +0.021) and 0.051 on unseen paraphrases (+0.029 to +0.073), with an invented-enum rate of 1.7% against 11%; C uses 48× fewer input tokens and, as trained, emitted tool calls on all 100 general-question probes.

4. Related work and what this study adds

Closest prior work. Shemla et al. (2026) fine-tune Gemma-4-E4B and Qwen3-4B with 8-bit QLoRA on ~1,700 examples over AssetOpsBench's 23 MCP tools and show that description-free inference — no tool catalog in the prompt — outperforms an unfine-tuned schema-informed baseline while cutting prompt tokens by 94.7%. That is the same headline as §3.2, reported first. Their evaluation is 30 held-out scenarios, single training runs, structural F1 plus an LLM judge; their discussion names two mechanisms — context competition and schema parsing at inference time — and states that they do not isolate them, listing as future work evaluating fine-tuned models with and without schemas in context, multi-seed runs with confidence intervals, and larger benchmarks. This study addresses several of those open questions: the single-schema diagnostic (§3.1) separates the prompt's effect from the tools'; arm D (§3.3) is fine-tuning *with* the schema present, matched to the schema-free arm on data at 4 tools (three seeds) and 64 tools (one seed); the catalog scales to 64 tools with up to 3,200 held-out queries per condition (3,122 for unseen paraphrases); seen and unseen paraphrase sets measure seen-to-unseen transfer; the GPT-OSS adapter configurations have 3 seeds (D at 64 tools has one; Inkling-Small has two) and the headline has tool-clustered intervals; rank goes down to 1. They also measured what we did not — general-capability retention after fine-tuning, where they found 18–39% degradation on multiple-choice reasoning.

Framing. Soni et al. (IAAI 2026, "Schema on the Inside") internalize a database schema into an 8B model for text-to-SQL in production, replacing a 17k-token schema prompt with under 100 tokens at higher accuracy than a prompted frontier model.

Parametric tool representations. ToolGen (Wang et al., ICLR 2025) represents each of ~47k tools as a vocabulary token and generates tool calls as next-token pre-

diction; Toolscaler (Su et al., EMNLP Findings 2025) replaces one-token-per-tool with structure-aware semantic codes so similar tools share subtokens; ToolkenGPT (Hao et al., 2023) learns tool embeddings for a frozen model. ParaTool (Yu et al., ICML 2026) encodes each tool as its own LoRA module and composes them with a learned gate, and finds that a single shared LoRA over thousands of tools performs poorly (§3.6). These target open catalogs of thousands of tools; this study targets the fixed-catalog regime with one ordinary adapter and no vocabulary changes, and measures where that simpler design holds.

Diagnosing what parametric tool knowledge actually contains. ToolSense (Hathidara et al., 2026) generates realistic queries at three ambiguity tiers plus factual probes for any tool catalog, and finds that parametric-retrieval models that score well on fully-specified queries collapse by 0.50–0.64 on realistic ones — a knowledge-retrieval dissociation. The seen/unseen paraphrase split here (§3.5) and the format-vs-vocabulary finding (natural-only adapters emit the right JSON shape and invent the values) are the same distinction measured different way.

API fine-tuning and retrieval. Gorilla (Patil et al., 2023) fine-tunes on API documentation with retrieval-aware training; ToolLLM (Qin et al., 2024) trains on 16k real APIs with a neural retriever; Toolformer (Schick et al., 2023) self-supervises API insertion. These supply tool information at inference through retrieval; this study compares against a lexical retriever only (§3.1), not a trained one.

What this study adds: controlled measurement of the fixed-catalog question across catalog size, model size, adapter rank: the full-catalog penalty decomposed by failure class and shown not to be a property of the tools; a with-schema fine-tuning arm matched on training data; a lexical-retrieval baseline; a seen-to-unseen paraphrase transfer measurement; per-tool analysis; and variance reported at the level a reader can check.

5. Limitations

- **One retriever.** The retrieval baseline is a word-overlap (IDF) retriever with no training. A dense or fine-tuned retriever would likely close some of the paraphrase gap (recall@4 on unseen was 0.85); the adapter's margin over the *best* retrieval configuration is not measured;
- **Selection was easy.** One tool per domain, no get/set/list families. Adapters' `tool_ok` was ~0.99 at 64 tools. Real catalogs are harder on selection;
- **D at 64 tools is one seed** (§3.3); the matched comparison at 4 tools has three;
- **Validation/test.** Natural held-out selected the schedule; unseen paraphrases share underlying calls with it;
- **Position effect measured on one model's base checkpoint** (§3.1); not repeated for Inkling-Small or for the adapters;
- **Single turn, single call, exact match, temperature 0.** No tool results fed back, no chaining, no pass@k;
- **Synthetic everything;** paraphrases from one generator family. No human-written test set;
- **Limited study of larger model.** Largest was Inkling-Small: two seeds, schedule carried from GPT-OSS (converged by step 300, flat tail). The natural-only D

at 64 tools is one seed at one epoch;
- **Metric interacts with model style** (Inkling's optional-filling); both exact and lenient are reported.
- **No call detection is not evaluated; the adapter was not trained for it** (§3.8).

6. AI assistance and author contributions

This study was directed and executed by the author, who designed the arms and catalog-scaling protocol, ran and paid for all experiments, verified every reported number against per-response records, and made all decisions about scope and claims. An AI assistant (Claude, Anthropic and ChatGPT, OpenAI) drafted most of the pipeline scripts, evaluation harness, and analysis code, that the author reviewed and revised. An AI-based internal review process contributed corrections, text improvements and proposed the counterbalanced-order analysis, the retrieval baseline, the matched with-schema comparison and the tool-clustered intervals. The author is responsible for all claims and errors. Interaction records were retained.

Appendix A — the 8-seed grid at the pre-selection schedule

4 tools, 2e-4 constant, GPT-OSS-20B, 8 seeds. Natural / unseen.

config	2e-4 constant, 8 seeds	final schedule, 3 seeds	Δ natural / unseen (final − constant)
ex100_C	0.897±.044 / 0.322±.047	0.955±.011 / 0.345±.015	+0.058 / +0.023
ex100_D	0.980±.008 / 0.531±.052	0.982±.006 / 0.438±.063	+0.002 / **−0.093**
mix50_C	0.920±.037 / 0.653±.060	0.933±.019 / 0.650±.021	+0.013 / −0.003
mix100_C	0.938±.027 / 0.672±.080	0.957±.012 / 0.718±.047	+0.019 / +0.046

Seed SD is 2–4× smaller under the final schedule for the no-schema configs. Means moved by up to 0.06 in either direction; D's unseen accuracy is 0.093 lower at 3 seeds under the final schedule (per-seed 0.495, 0.350, 0.470) than at 8 seeds under the old one — a difference this study has too few seeds to interpret.

Appendix B — per-seed results, final schedule

Natural / unseen, GPT-OSS-20B unless noted.

config	seed 0	seed 1	seed 2
cat4 ex100_C	0.940 / 0.340	0.965 / 0.330	0.960 / 0.365
cat4 ex100_D	0.980 / 0.495	0.990 / 0.350	0.975 / 0.470
cat4 mix50_C	0.920 / 0.628	0.960 / 0.643	0.920 / 0.678
cat4 mix50_D	0.960 / 0.740	0.985 / 0.745	0.985 / 0.760
cat4 mix100_C	0.970 / 0.775	0.960 / 0.720	0.940 / 0.660
cat16 ex100_C	0.966 / 0.483	0.971 / 0.493	0.974 / 0.462
cat16 ex100_D	0.975 / 0.613	0.994 / 0.614	0.978 / 0.581
cat16 mix50_C	0.950 / 0.731	0.964 / 0.761	0.965 / 0.711

config	seed 0	seed 1	seed 2
cat16 mix100_C	0.969 / 0.799	0.974 / 0.786	0.984 / 0.794
cat64 ex100_C	0.953 / 0.414	0.956 / 0.401	0.951 / 0.432
cat64 ex100_D (1 epoch)	0.975 / 0.571	—	—
cat64 mix50_D	0.962 / 0.786	—	—
cat64 mix50_C	0.953 / 0.756	0.950 / 0.731	0.950 / 0.721
cat64 mix100_C	0.966 / 0.742	0.975 / 0.777	0.964 / 0.756
cat64 mix50_C rank 1	0.947 / 0.685	0.944 / 0.687	0.945 / 0.673
cat64 mix50_C rank 4	0.939 / 0.700	0.946 / 0.709	0.950 / 0.695
cat64 mix50_C rank 8	0.949 / 0.714	0.946 / 0.734	0.942 / 0.715
cat64 mix50_C, Inkling-Small	0.957 / 0.830	0.961 / 0.823	—

Held-out counts: natural 200 / 800 / 3,200; unseen 199–200 / 795 / 3,122 at 4 / 16 / 64 tools.

Appendix C — the 12 tools hit hardest by the full catalog, GPT-OSS-20B

tool	single schema	full catalog	penalty	adapter (mix50)
log_cheese_cave_entry	1.00	0.40	−0.60	0.89
create_wheel_thrown_vessel	0.84	0.24	−0.60	0.96
calibrate_propane_regulator	0.78	0.20	−0.58	0.95
inspect_bell_mechanism	0.86	0.28	−0.58	0.97
list_seismic_events	0.96	0.38	−0.58	0.95
report_contest_qso	0.86	0.28	−0.58	0.99
schedule_bonsai_defoliation	0.84	0.26	−0.58	0.99
assign_sail_maintenance_crew	0.66	0.10	−0.56	0.85
calibrate_quarantine_sensors	1.00	0.46	−0.54	0.97
log_wither_progress	0.78	0.26	−0.52	0.95
adjust_typebar_strike	0.82	0.30	−0.52	0.81
create_dry_stone_wall	0.94	0.44	−0.50	0.98
mean, these 12	**0.862**	**0.300**		**0.939**
mean, other 52	0.897	0.648		0.954

Per-tool adapter accuracy correlates with the single-schema baseline ($\rho = +0.48$) and not with lexical similarity to the nearest other tool ($\rho = -0.13$).

References

- Hao, S., Liu, T., Wang, Z., Hu, Z. (2023). ToolkenGPT: Augmenting frozen language models with massive tools via tool embeddings. *NeurIPS*.
- Hathidara, A., Sistla, S. S., Schreiber, S., Bansal, S. (2026). ToolSense: A diagnostic framework for auditing parametric tool knowledge in LLMs. arXiv:2606.12451.

- Patil, S. G., Zhang, T., Wang, X., Gonzalez, J. E. (2024). Gorilla: Large language model connected with massive APIs. *NeurIPS*.
- Qin, Y. et al. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *ICLR*.
- Schick, T. et al. (2023). Toolformer: Language models can teach themselves to use tools. *NeurIPS*.
- Schulman, J. et al. (2025). LoRA without regret. Thinking Machines Lab blog.
- Shemla, Y., Yakobe, A., Agarwal, T., Patel, D., El Maghraoui, K. (2026). Internalizing tool knowledge in small language models via QLoRA fine-tuning. arXiv:2605.17774.
- Soni, C., Chourasia, S., Kumar, G., Kapoor, H. (2026). Schema on the inside: A two-phase fine-tuning method for high-efficiency text-to-SQL at scale. *AAAI/IAAI-26*, 40(47), 40110–40117.
- Su, Y. et al. (2025). Toolscaler: Scalable generative tool calling via structure-aware semantic tokenization. *Findings of EMNLP 2025*, 556–578.
- Wang, R., Han, X., Ji, L., Wang, S., Baldwin, T., Li, H. (2025). ToolGen: Unified tool retrieval and calling via generation. *ICLR*.
- Yu, Z., Meng, Q., Chu, Q., Hao, Y., Shi, C., Yang, C. (2026). ParaTool: Shifting tool representations from context to parameters. *ICML*.